Session Walkthrough: LLM-Based Speech Classification

Canadian Hansard, 1920–1949

Mathias Bühler

2026-05-19

The big picture

By the end of our last session we had two clean datasets:

- `output/speeches_all.csv` — every Hansard speech, with stemmed/cleaned text from `prepare_claude_v2.py`.
- `output/MPs.csv` — a biographical panel of MPs scraped from ParlInfo (name, party, riding, province, term dates, gender).

Today's session built an end-to-end LLM-based classification pipeline on top of those datasets. The arc went from "we have speeches and biographies sitting in separate files" to "every speech is labelled with a topic and we can plot how Parliament's attention shifted across the 1920s–40s." There were three pieces, run in order:

1. Merge the speeches with MP biographical data, so we know who said what.
2. Use Gemini (a large language model) to classify each speech into one of ten topics.
3. Inspect and visualise the results.

1 Step 1 — Merge speeches with MP biographies

Script: `src/merge_speeches_mps.py`

The hard part of this step is that the two files share **no numeric ID**. Speeches list a `speakername` like `"Mr. William Lyon Mackenzie King:"`; MPs list `"King, William Lyon Mackenzie"`. So we have to match on the name itself.

What the script does, in order:

1. **Normalise names into a join key.** The function `to_join_key` strips honorifics (*Mr*, *Hon*, *Dr*, ...), strips accents (*é* becomes *e*), lowercases, and reformats `"First Middle Last"` into `"last, first middle"`. We also repair "mojibake" — characters that got mangled by being saved with the wrong encoding (`ÃĆ` instead of `é`).
2. **Deduplicate the MP panel.** Some MPs appear twice in the same year (re-elections, by-elections). We keep the row with the widest term span so a speech is never matched to two MPs.
3. **Exact merge on `(name_join, year)`.** Most speeches match here.
4. **Fuzzy fallback for the rest.** Using the `rapidfuzz` library, we score residual unmatched names against the MPs serving in that year. To accept a fuzzy match we require:

- similarity score ≥ 88,
- the second-best candidate is at least 5 points worse (so the match is unambiguous), and
- the **surnames** alone score ≥ 85 (this prevents weird first-name-only matches).

5. **Drop the unmatched.** Speeches we cannot attribute to an MP are dropped rather than carried with missing covariates.

Output: `output/speeches_all_mps.csv`, with speech metadata and MP attributes on every row. The script also prints coverage by decade and the 20 most-dropped speaker names so you can sanity-check.

Why this is the right shape of solution. Without a shared ID, fuzzy matching is unavoidable. The discipline is to: (a) define a normalisation step that catches the systematic differences (honorifics, accents, name order); (b) do the exact merge first because it is cheap and unambiguous; and (c) only fall back to fuzzy matching with strict thresholds for what is left.

2 Step 2 — Classify each speech with an LLM

Script: `src/classify_speeches_gemini.py`

The KMeans clusters from last session were hard to interpret. The clusters were defined by word co-occurrence, not by meaning, so a cluster might mix unrelated topics that happened to share vocabulary. We swap that out for an LLM that reads each speech and picks a label from a fixed list of ten:

> *Wartime, Infrastructure, Health care, Education, Income and Tax, International relations, Budget, Trade, Immigration, Other.*

Key design choices

1. **Use the raw (unstemmed) text.** The LLM understands English. Stemming `"taxation"` into `"taxat"` actively hurts it. So we re-join `speeches_all_mps.csv` against the original `input/Speeches*.xlsx` on `basepk` (a stable speech ID that survives all our processing) to recover the unstemmed text and ParlInfo's own topic tags.

2. **Cache the merged panel as parquet.** Reading the 12 Excel files takes about 2.5 minutes. We do it once, write `output/speeches_with_raw.parquet`, and load that in about 3 seconds on later runs. Use `--rebuild-cache` to refresh.

3. **Constrain the model's output with a schema.** We define a Python `Enum` of the ten categories and pass it as `response_schema`. Gemini is then physically incapable of returning anything other than one of the ten strings. No regex parsing, no "Other" fallback for malformed output. This is the single most important trick:

```
class Category(str, enum.Enum):
    WARTIME = 'Wartime'
    INFRASTRUCTURE = 'Infrastructure'
    HEALTH_CARE = 'Health care'
    EDUCATION = 'Education'
    INCOME_AND_TAX = 'Income and Tax'
    INTERNATIONAL_RELATIONS = 'International relations'
```

```
    BUDGET = 'Budget'
    TRADE = 'Trade'
    IMMIGRATION = 'Immigration'
    OTHER = 'Other'

GEN_CONFIG = types.GenerateContentConfig(
    system_instruction=SYSTEM_INSTRUCTION,
    temperature=0.0,
    response_mime_type='text/x.enum',
    response_schema=Category,
)
```

4. **Write a careful system prompt.** The category definitions distinguish boundary cases that matter for our period. For example, tariffs as *revenue* go under "Income and Tax"; tariffs as *trade policy* go under "Trade". Without that distinction, the model would mix them.

5. **Run asynchronously with a concurrency cap.** Each API call takes about 0.6 s. If we did them in sequence, 162,000 speeches would take 27 hours. With `asyncio.Semaphore(80)` (80 in-flight requests at a time), it finishes in roughly 30 minutes.

6. **Append-as-you-go, resumable.** Every 500 results we flush to `output/speeches_classified.csv`. When you re-run the script, it reads the existing CSV, drops the `basepk`s already done, and only classifies the remainder. `Ctrl-C` is safe; the cost of a crashed run is bounded.

7. **Retry with backoff on rate-limit errors.** Google's free tier rate-limits aggressively. We catch `429 / RESOURCE_EXHAUSTED` and back off 30 s, 60 s, 120 s, ... with jitter.

To replicate

```
pip install google-genai pandas openpyxl matplotlib rapidfuzz
echo "GEMINI_API_KEY=your_key_here" > .env
# get key at https://aistudio.google.com/apikey

# pilot run on a stratified sample
python src/classify_speeches_gemini.py --sample 500

# full run (about 30 min, a few dollars)
python src/classify_speeches_gemini.py
```

Output: `output/speeches_classified.csv` with columns `basepk, category, error`.

3 Step 3 — Inspect the classifications

Script: `src/inspect_classifications.py`

This is the validation step. We never trust an LLM's output without checking it against something independent. Four things:

1. **Cross-tab against ParlInfo's own `maintopic`.** For each Gemini category we print the top 3 ParlInfo maintopics. If Gemini's "Wartime" lines up with ParlInfo's "War" or "Defence", that is confirmation. If it lines up with random topics, something is wrong.

2. **Plot category share and counts over time.** Two plots: a stacked-area share plot (`output/category_share_by`) and an absolute-count line plot with the WWII window shaded (`output/category_count_by_year.png`). The latter makes the wartime surge in 1939–45 jump out.

3. **Plot category share by party.** Restricted to the top 5 parties so the bar chart stays readable. Tells us which topics each party talked about most.

4. **Spot-check 3 random speeches per category.** Prints the first 280 characters of three random speeches per label, with a fixed `random.seed(42)` so the picks are reproducible. This is the cheapest and most honest validation: read the speech, ask yourself if you would have given it that label.

4 How to replicate end-to-end

From a fresh clone, assuming `input/Speeches*.xlsx` and `input/Link_ID.csv` are in place:

```
# one-time, from prior sessions
python src/scrape_parlinfo.py # -> output/MPs.csv
python src/prepare_claude_v2.py # -> output/speeches_all.csv

# today's work
python src/merge_speeches_mps.py # -> output/speeches_all_mps.csv
python src/classify_speeches_gemini.py # -> output/speeches_classified.csv
python src/inspect_classifications.py # -> plots and console diagnostics
```

5 The three ideas worth remembering

1. **When IDs do not exist, build a normalised join key and use exact-then-fuzzy.** Fuzzy alone is too permissive; exact alone leaves too much on the floor.

2. **Constrained generation beats prompt engineering.** Forcing the model's output to conform to an `Enum` is more reliable than asking it nicely to "respond with one of the following ten labels".

3. **Validate LLM output with at least two independent checks.** We used (a) cross-tab against a different labelling (ParlInfo's `maintopic`) and (b) human spot-checks. Neither alone is enough.